

Rajalakshmi Engineering College

Name: Naren S
Email: 241901066@rajalakshmi.edu.in
Roll no: 241901066
Phone: 6382463115
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Raj is solving a physics problem involving projectile motion, where he needs to calculate the time a ball hits the ground using a quadratic equation of the form $ax^2 + bx + c = 0$. Depending on the coefficients, the ball may hit the ground once, twice, or not at all in real time.

Help Raj find all real roots of the equation, if any.

Note: discriminant = $b^2 - 4ac$

Input Format

The input consists of three space-separated doubles a, b, and c, representing the coefficients of the quadratic equation.

Output Format

If there are two real roots, print:

- "Two real solutions:"
- "Root1 = <value>"
- "Root2 = <value>"

If there is one real root, print:

- "One real solution:"
- "Root = <value>"

If there are no real roots, print:

- "There are no real solutions."

Note: values are rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1 6 9

Output: One real solution:

Root = -3.00

Answer

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double a = sc.nextDouble();
        double b = sc.nextDouble();
        double c = sc.nextDouble();
        double d = b * b - 4 * a * c;
        if (d > 0) {
            double r1 = (-b + Math.sqrt(d)) / (2 * a);
            double r2 = (-b - Math.sqrt(d)) / (2 * a);
            System.out.printf("Two real solutions: Root1 = %.2f Root2 = %.2f", r1, r2);
        } else if (d == 0) {
```

```
double r = -b / (2 * a);
System.out.printf("One real solution: Root = %.2f", r);
} else {
    System.out.print("There are no real solutions.");
}
}
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Ted, the computer science enthusiast, has accepted the challenge of writing a program that checks if the number of digits in an integer matches the sum of its digits.

Guide Ted in designing and writing the code to solve this problem using a 'do-while' loop.

Input Format

The input consists of an integer N, representing the number to be checked.

Output Format

If the sum is equal to the number of digits, print "The number of digits in N matches the sum of its digits."

Else, print "The number of digits in N does not match the sum of its digits."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 20

Output: The number of digits in 20 matches the sum of its digits.

Answer

```
import java.util.Scanner;
```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        int num = N;
        int sum = 0;
        int digits = 0;

        do {
            int digit = num % 10;
            sum += digit;
            digits++;
            num /= 10;
        } while (num > 0);

        if (sum == digits) {
            System.out.print("The number of digits in " + N + " matches the sum of its
digits.");
        } else {
            System.out.print("The number of digits in " + N + " does not match the
sum of its digits.");
        }

        sc.close();
    }
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

John is a fitness trainer, and he wants to use the BMI calculator to assess the body mass index of his clients. He has a list of clients based on their height and weight.

John plans to write a program to quickly determine the BMI and provide a classification for each client.

If BMI is less than 18.5, the program will classify it as "Underweight" If BMI

is between 18.6 and 24.9, the program will classify it as "Normal Weight" If BMI is between 25.0 and 29.9, the program will classify it as "Overweight" If BMI is 30.0 or higher, the program will classify it as "Obese"

Note: Formula to calculate BMI = $\text{weight}/(\text{height}*\text{height})$

Input Format

The first line of input consists of a double value, representing the height of the person in meters.

The second line consists of a double value, representing the weight of the person in kilograms.

Output Format

The first line of output prints "BMI: " followed by a double (rounded to two decimal places) representing the calculated BMI.

The second line prints "Classification: " followed by a string indicating the BMI category (Underweight, Normal Weight, Overweight, or Obese).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1.2
45.2

Output: BMI: 31.39
Classification: Obese

Answer

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double height = sc.nextDouble();
        double weight = sc.nextDouble();

        double bmi = weight / (height * height);
```

```
System.out.printf("BMI: %.2f ", bmi);

if (bmi < 18.5) {
    System.out.print("Classification: Underweight");
} else if (bmi >= 18.6 && bmi <= 24.9) {
    System.out.print("Classification: Normal Weight");
} else if (bmi >= 25.0 && bmi <= 29.9) {
    System.out.print("Classification: Overweight");
} else {
    System.out.print("Classification: Obese");
}

sc.close();
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Joe has a favourite number, let's call it X. He wants to check if X is divisible by the sum of its digits. If it is, he considers it a lucky number. If not, he wants to find the closest smaller number, that is divisible by the sum of digits of X. Joe has challenged his friends to solve this puzzle at his birthday party.

Example

Input:

157

Output:

157 is not divisible by the sum of its digits.

The closest smaller number that is divisible: 156

Explanation:

The sum of the digits of X is $1+5+7=13$. Since 157 is not divisible by 13, we

need to find the closest smaller number that is divisible by 13. 156 is divisible by 13, it is the closest smaller number that meets the requirement.

Input Format

The input consists of an integer X, representing Joe's favourite number.

Output Format

If X is a lucky number, then the output must be in the format: "X is divisible by the sum of its digits."

If not, then the output must be in the format:

"X is not divisible by the sum of its digits.

The closest smaller number that is divisible: Y",

where X is the entered number and Y is the closest number.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 120

Output: 120 is divisible by the sum of its digits.

Answer

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int X = sc.nextInt();
        int temp = X, sum = 0;
        while (temp > 0) {
            sum += temp % 10;
            temp /= 10;
        }
        if (X % sum == 0) {
            System.out.print(X + " is divisible by the sum of its digits.");
        }
    }
}
```

```
    } else {  
        int Y = X - 1;  
        while (Y > 0 && Y % sum != 0) {  
            Y--;  
        }  
        System.out.print(X + " is not divisible by the sum of its digits. The closest  
smaller number that is divisible: " + Y);  
    }  
}
```

Status : Correct

Marks : 10/10